Genetic Programming and Evolvable Machines (2019) 20:5–35
https://doi.org/10.1007/s10710-018-9339-y

# DENSER: deep evolutionary network structured representation

**Filipe Assunção[1] · Nuno Lourenço[1] · Penousal Machado[1] · Bernardete Ribeiro[1]**

Received: 22 May 2018 / Revised: 7 September 2018 / Published online: 27 September 2018


## Abstract

Deep evolutionary network structured representation (DENSER) is a novel evolutionary approach for the automatic generation of deep neural networks (DNNs) which combines the principles of genetic algorithms (GAs) with those of dynamic structured grammatical evolution (DSGE). The GA-level encodes the macro structure of evolution, i.e., the layers, learning, and/or data augmentation methods (among others); the DSGE-level specifies the parameters of each GA evolutionary unit and the valid range of the parameters. The use of a grammar makes DENSER a general purpose framework for generating DNNs: one just needs to adapt the grammar to be able to deal with different network and layer types, problems, or even to change the range of the parameters. DENSER is tested on the automatic generation of convolutional neural networks (CNNs) for the CIFAR-10 dataset, with the best performing networks reaching accuracies of up to 95.22%. Furthermore, we take the fittest networks evolved on the CIFAR-10, and apply them to classify MNIST, Fashion-MNIST, SVHN, Rectangles, and CIFAR-100. The results show that the DNNs discovered by DENSER during evolution generalise, are robust, and scale. The most impressive result is the 78.75% classification accuracy on the CIFAR-100 dataset, which, to the best of our knowledge, sets a new state-of-the-art on methods that seek to automatically design CNNs.

**Keywords** Automated machine learning · NeuroEvolution · Deep neural networks · Convolutional neural networks · Dynamic structured grammatical evolution

✉ Filipe Assunção
fga@dei.uc.pt

Nuno Lourenço
naml@dei.uc.pt

Penousal Machado
machado@dei.uc.pt

Bernardete Ribeiro
bribeiro@dei.uc.pt

[1] CISUC - Department of Informatics Engineering, University of Coimbra, Coimbra, Portugal

## 1 Introduction

In our daily life, even without noticing, we interact with machine learning (ML) [10] systems: the results of web search and recommendation systems; translation and speech recognition tools; e-mail categorisation, spam and malware detection; abnormal behaviour detection (e.g., fraudulent transactions, or video surveillance); credit risk assessment; plagiarism detection; customer behaviour analysis and segmentation.

To build effective (and high performing) ML systems we need data of quality, and for this we can count on the ever growing amount of information that is constantly collected nowadays. For example, retail stores collect all the items we ever bought, and when and where we purchased them; websites store our movement such as the landing page, and how long we navigate each page; search engines save all our queries; governments share open data datasets; cable box providers collect our viewing habits (as do e-readers with our reading preferences); many of us openly share our daily-life events on social networks, where in addition to photos, text, and sentiments, we often share our location.

In traditional ML approaches, even before the training of the model, there is the need to pre-process the dataset and design features (i.e., dataset properties) that are used to guide the system towards the learning of the desired task. As an example, if our goal is to develop a ML model to suggest the reselling price of an automobile we can consider the maker, model, engine size, horsepower, consumption, mileage, and age of the automobile. However, designing features that enable a system to perform more challenging tasks (e.g., object detection or speech translation) is not so straightforward. This happens because expertise and domain specific knowledge are needed. Additionally, features tend to be problem dependent, i.e., for each different task different characteristics have to be extracted.

To overcome the hand-craft iterative process of designing and testing features every time we want to deploy a new ML system, we can opt for methods that automatically learn the best data representation. Deep learning (DL) methods [18] are an example of these type of techniques. They receive the raw input (e.g., pixel values of an image), and are structured by layers: each layer successively applies transformations to the previous, to generate increasingly more abstract representations of the raw input signal. The learned representation is then used for solving the task at hand.

Despite the wide range of applications of ML and DL, their models require an extensive parameterisation stage. Similarly to the feature extraction, the best model is often found iteratively, by trial-and-error. Evolutionary computation (EC) methods are known to be an effective optimisation tool [5], and thus their application to the automatic search of near-optimal parameterisations of DL models is natural.

The current paper is an extension of Assunção et al. [3], and describes DENSER, a novel and general purpose framework for the automatic generation of deep neural networks (DNNs). Section 2 sets the motivation for developing

DENSER. Section 3 briefly surveys the most important and influential neuroevolution (NE) works, with special focus on those that tackle the evolution of DNNs. Section 4 describes dynamic structured grammatical evolution (DSGE) which serves as base for developing DENSER (detailed in Sect. 5). Section 6 presents the conducted experiments. In Sect. 7 conclusions are drawn and future work is addressed.

## 2 Motivation

The search for approaches to automatically configure ML methods is not new, and is known as automated machine learning (AutoML). The main goal of these approaches is to create and configure ML systems without the need for human intervention. This would remove the need of domain knowledge and expertise, and alleviate the burden of an extensive iterative trial-and-error search. The ultimate goal of AutoML is to empower non-expert users with ML systems that can be used off-the-shelf. Given the current demand for such systems there are even competitions, such as ChaLearn [21, 22], where the goal is to build models to tackle ML tasks with no manual parameterisation.

AutoML has four key automation goals: (i) data pre-processing; (ii) feature engineering; (iii) model selection; and (iv) hyperparameter configuration. Along with manual search, the most used method for optimising algorithmic parameterisations is grid search, which performs an exhaustive search of the discrete parameter's domain. Duan and Keerthi [13] use grid search for optimising the regularisation and kernel parameters of support vector machines (SVMs) with a Gaussian kernel; Jiménez et al. [26] propose two focused grid search methods[1] and apply them to the optimisation of the parameters of SVMs and multilayer perceptrons (MLPs).

One of the main advantages of grid search is that it is highly parallelisable. Nonetheless, it requires huge computational power, as all the parameter combinations in the grid have to be tested; further, it does not deal with continuous values (that must be discretised). Bergstra et al. [8, 9] show that given the same computational time, random search is able to discover a better parameterisation for an artificial neural network (ANN) than grid search.

In contrast with the previous methods, that make no assumptions about the domain to be explored, we have probabilistic methods. An example of such is Bayesian optimisation [40]—the objective is to model, probabilistically, the behaviour of the system to be tuned, so that we drive search towards regions of the domain that are prone to generate better results. Snoek and Larochelle [53] demonstrate the ability of Bayesian optimisation to tune the parameters of the Branin-Hoo function, Logistic Regression, Online Latent Dirichlet Allocation, Latent Structured SVMs, and convolutional neural networks (CNNs).

Notwithstanding the relevance of the previous works, none of them tries to tackle, simultaneously, all previously enumerated key automation goals of AutoML.

[1] Grid search methods that adapt the resolution of the grid in run-time.

Examples of frameworks that do so are Auto-WEKA [60], Hyperopt-Sklearn [31], and Auto-Sklearn [15]. These frameworks use Bayesian optimisation to discover pipelines to automate ML tasks (from data preprocessing, to the model selection and parameterisation). As the frameworks names suggest, they are based on the selection and parameterisation of the primitives available in the Weka [64], and Scikit-learn [45] libraries.

In addition to Bayesian optimisation, EC has also been successfully applied to the automatic parameterisation of ML algorithms. Examples of such are the tuning of SVM parameters [12], the weight of the samples to be used by the k-Nearest Neighbours [28], or of multiple parameters of ANNs (e.g., number of neurons, layers, learning rate) [67]. Despite the many scenarios where EC is applied to solve particular optimisation problems, there are also approaches, such as CMA-ES [23], that work as general purpose numerical optimisers. In the particular case of ML we highlight tree-based pipeline optimization tool (TPOT), which is in line with Auto-Weka, Hyperopt-Sklearn, and Auto-Sklearn, but allows the generation of pipelines of unrestricted size.

With the increase in the availability of GPU computing, DL models have seen a rise in popularity. The current work seeks to propose a general purpose framework that is capable of automatically generating off-the-shelf DNNs, i.e., a method that searches for the best structure, and its parameters. In challenges such as the ChaLearn, the system receives as input the features; However, the features are not necessarily easy to design. DL methods are known for learning representation, and consequently, are automatic feature extractors. We will focus on the use of EC; the key reasons for selecting EC instead of grid search, Bayesian optimisation, or any other optimisation approach are two: (i) EC is highly parallelisable, and thus easily scalable; (ii) EC is flexible and dynamic, which facilitates its application to a wide set of different network topologies and problems. The next section briefly surveys NE methods—approaches that apply EC techniques to the automatic generation of ANNs. Special focus will be given to the search of deep architectures; the challenges related to the evolution of DNNs are also discussed.

## 3 Neuroevolution

In general, to design effective ANNs one has to: (i) pre-process the data; (ii) extract and select features; (iii) decide the structure of the network, e.g., number of hidden-layers, number of units in each layer, and activation functions; and (iv) choose the learning algorithm, and optimise the correspondent parameters (e.g., learning rate, or momentum). There are plenty of evolutionary approaches that seek to solve each of these steps; Verbancsics and Harguess [62] apply HyperNEAT [56] to feature learning, but it under-performs in classification tasks by itself; Yu and Bir [68] and Schuller et al. [49] automatically extract features for facial and speech emotion recognition, respectively; Xue et al. [66] survey different evolutionary approaches to feature selection. Despite the numerous works on data manipulation for later use by ML approaches, our focus is towards DL models, which are able to automatically extract features. Thus, we concentrate on the

automatic optimisation of the ANNs structure and respective parameterisation, in a field known as neuroevolution (NE) [16].

NE approaches are commonly grouped according to the aspects of the ANNs that they optimise: (i) learning [17, 42, 46]; (ii) topology [24, 48, 55]; or (iii) both topology and learning [57, 61, 63], in what is known as topology and weight evolving artificial neural network (TWEANN). In NE the candidate solutions encode the ANNs with the parameters that are being optimised. The quality of the candidate solutions is based on how well each ANN performs in a given problem (e.g., classification accuracy on the CIFAR-10 dataset).

The vast majority of NE works target the evolution of small networks for very specific tasks. With DENSER our goal is to evolve large scale networks that can deal with vast amounts of data and challenging tasks. As an example, consider the VGG network [52]: a 16 to 19 deep CNN that is often used for object recognition tasks. The number of neurons and connections involved in deep architectures usually turns connection [14, 30, 34] or node-based [4, 41, 57] evolutionary methods impractical for discovering high performing networks, due to the large search space that needs to be scanned. Therefore, for evolving deep networks practitioners often resort to layer-based encodings [27, 38, 59]. For similar reasons, it is unfeasible to directly evolve the weights of the networks, which easily reach the range of thousands, or even millions of parameters; when the training of the networks is optimised using EC usually only the hyper-parameters are tuned and the networks trained using gradient-descent algorithms [38, 59].

Loshchilov and Hutter [35] developed an approach based on Evolutionary Strategies to optimise the hyper-parameters of deep networks; the tuned parameters are concerned with the topology (e.g., number of filters in the convolution layers, and number of neurons in fully-connected layers) and learning (e.g., batch size, and learning rates). This approach requires the *a-priori* definition of the network structure that may be suitable for solving the problem, and consequently there is no optimisation of the sequence of layers and connections between them.

The idea of optimising hyper-parameters for deep networks is further extended in coevolution deepNEAT (CoDeepNEAT) [38], where the structure of the network is searched combining the ideas behind symbiotic, adaptive neuro-evolution (SANE) [41] and neuroevolution of augmenting topologies (NEAT) [57]. Two populations are evolved in simultaneous: one of modules and another one of blueprints, which specify the modules that should be used. Learning and data augmentation parameters are also optimised.

A similar approach is proposed in CGP-NN [59], where cartesian genetic programming (CGP) [39] is applied to the evolution of the architecture of CNNs. However, instead of automatically searching the modules, they are specified by the user, and just their combination and parameterisation is evolved.

In this work we want to make the automatic generation of deep networks as easy and transparent as possible. That is the reason why we adopt a grammar-based approach. Grammars allow us to specify different network types, such as AutoEncoders or CNNs, without the need to change any implementation details. Further, grammar-based methods make it easy to incorporate knowledge,

allowing the specification of modules and/or parameters that we may know or suspect that work well on certain problem domains.

There are several approaches concerned with the evolution of ANNs using grammatical evolution (GE) [44]. The majority of them focus on the tuning of a single hidden-layer, i.e., on the number of neurons and their connections from the input to the hidden-nodes and from the hidden-nodes to the outputs [2, 3, 55]. Assunção et al. [4] describe a grammar-based method that is able to evolve ANNs with more than one hidden-layer. Despite theoretically suited to generate deep networks, the involved amount of neurons and connections make the domain space too large to be searched within an acceptable time. Motivated by the previous, Baldominos et al. [7] propose a GE NE layer-based method; the defined grammar uses apriori knowledge; and the method suffers from the problems commonly pointed out to GE: locality and redundancy [37].

## 4 Dynamic structured grammatical evolution

Dynamic structured grammatical evolution (DSGE) [4, 36] is a variant of GE [44], that is based on structured grammatical evolution (SGE) [37]. In this section we evidence the differences between GE, SGE, and DSGE. The three methods are based on rewriting systems that are formally defined by a 4-tuple: $G = (N, T, P, S)$, where: (i) $N$ is the set of non-terminal symbols; (ii) $T$ is the set of terminal symbols; (iii) $P$ is the set of production rules of the form $x ::= y$, $x \in N$ and $y \in N \cup T^*$; and (iv) $S$ is the initial symbol. The differences between GE, SGE, and DSGE mainly concern the representation of the candidate solutions; in particular, on how the derivation steps needed for encoding the candidate solutions are stored. Therefore, the decoding procedures may also differ.

### 4.1 Grammatical evolution

The candidate solutions in GE are encoded as variable-length strings of decimal numbers. Each integer represents a grammar derivation step. The decoding procedure reads each codon (i.e., integer) from left to right and selects the derivation step to be applied, using the following rule:

$$\text{rule} = \text{codon MOD non\_terminal\_possibilities},$$

where non_terminal_possibilities is the number of possibilities for the expansion of the current non-terminal symbol. If for a given non-terminal symbol there is only one possibility for its expansion no integer is read.

The used grammar can be recursive, and thus the number of codons in the genotype can be insufficient. To solve this issue, wrapping is applied, i.e., the genetic material is re-used, and the codons are re-read from the start of the genotypic material.

### 4.2 Structured grammatical evolution

The modulus mathematical operation used in GE to map a codon to the expansion possibility allows multiple codons to generate the same output, which can turn mutation ineffective because it is likely that a mutation in one of the codons generates no changes in the phenotype (high redundancy); on the other hand, a mutation can change too much of the phenotype when it is successful, since a change in one of the codons can impact the remaining of the expansion possibilities from that point onwards (low locality). The previous issues motivate SGE.

Similarly to GE, in SGE the genotype of a candidate solution is a string of integers. However, whilst in GE there is a single list of integers that is used for encoding the expansions of all non-terminal symbols, in SGE there is a separate list for every non-terminal symbol. The size of the list of integers for each non-terminal symbol is of the size of the maximum number of expansions for that specific non-terminal symbol[2]; each integer can assume a value between 0 and the number of expansion possibilities for that specific non-terminal symbol; this avoids using the modulus mathematical operation. To deal with recursion, a maximum depth is defined (similar to what happens in tree-based GP), and a set of intermediate symbols are created.

The decoding procedure follows the same rationale than the one described for GE, with the difference that the codons are not read sequentially from a single list of integers, but are rather read sequentially from the list associated with the non-terminal symbol that is to be expanded.

### 4.3 Dynamic structured grammatical evolution

The genotype in SGE encodes the maximum number of expansions for each non-terminal symbol; this does not mean that all the genotype is used in the decoding procedure, but rather that at most those codons can be used. Therefore, unless we store the codons that are in use, it is possible to apply genetic operators to non-coding genotype parts, which is not translated into any evolutionary improvement. The main goal of DSGE is to encode only the portions of the genotype that are needed to represent the candidate solutions, and the genotype grows as needed; the gain is three-fold: (i) all the genotype is used; (ii) there is no need to pre-process the grammar in order to compute the maximum number of expansions; (iii) it is easier to deal with recursion, as the genotype grows as needed, and thus no wrapping is necessary.

The genotype is encoded as in DSGE: a list of expansion possibilities for each non-terminal symbol; the decoding procedure of DSGE is the same of SGE; the generation of the initial population is different: while in SGE we initialise sequences with the maximum number of codons, in DSGE only the necessary ones are encoded.

[2] To compute the maximum number of expansions of each non-terminal symbol the grammar must be pre-processed; for further details check Section 3.1 of [37].

## 5 Proposed approach

Deep evolutionary network structured representation—DENSER—is a novel general purpose approach for the evolution of DNNs. It combines the principles of genetic algorithms (GAs) and DSGE to mitigate the issues pointed out in Sects. 2 and 3. We make an addition to DSGE to facilitate the encoding of real values: instead of encoding them as the outcome of the application of several production rules suit to generate floats we encode them directly, and then apply standard genetic operators to them (e.g., Gaussian mutation). The upcoming sub-sections respectively detail the representation, the genetic operators, and how the generated networks are evaluated.

### 5.1 Representation

Each candidate solution encodes a single DNN by means of an ordered sequence of evolutionary units, and the associated parameters; the representation facilitates the encoding of: (i) any type of layers and their respective parameters; (ii) the learning algorithm and parameters; and/or (iii) data augmentation approaches and parameters. The representation of the candidate solutions has two independent levels:

**GA level** Encodes the macro structure of the DNNs, and represents the sequence of evolutionary units that form the network; each unit in the sequence later serves as the starting non-terminal symbol for the expansion of the DSGE level genotype. This representation level requires the definition of the valid structure of the genotype, i.e., the goal of evolution: layers (which types of layers can be used, and in what order), learning, and data augmentation. This input is important in case the user wants to include any prior knowledge into the search space (e.g., sequences of layers that are known to work well). For example, CNNs are usually composed by convolution and/or pooling layers (responsible for representation learning, i.e., for learning the features), and fully-connected layers (that perform classification based on the learnt representation). A possible GA structure for the evolution of CNNs is then: [(features, 1, 10), (classification, 1, 2), (softmax, 1, 1), (learning, 1, 1)]; the numbers stand for the minimum and maximum number of layers of that type, respectively. The previous GA structure is able to encode CNNs with a minimum of 3 and a maximum of 14 layers: from 1 to 10 convolution and pooling (features) layers, followed by 1 or 2 fully-connected (classification) layers, and one softmax layer; the learning rule is for encoding the learning algorithm and its parameters.

**DSGE level** Whilst the GA level encodes the macro structure of the genotype, it is in the DSGE level that the parameters are stored. The parameters are codified in a backus-naur form (BNF) grammar, and are represented as ranges, or closed sets of possibilities. An example of a valid

```
<features> ::= <convolution>                                                    (1)
             | <pooling>                                                        (2)
<convolution> ::= layer:conv [num-filters,int,1,32,256] [filter-shape,int,1,1,5] (3)
             [stride,int,1,1,3] <pad> <activation> <bias>                       (4)
             <batch-normalisation> <merge-input>                                (5)
<batch-normalisation> ::= batch-normalisation:True                              (6)
             | batch-normalisation:False                                        (7)
<merge-input> ::= merge-input:True                                              (8)
             | merge-input:False                                                (9)
<pooling> ::= <pool-type> [kernel-size,int,1,1,5][stride,int,1,1,3] <pad>       (10)
<pool-type> ::= layer:pool-avg                                                  (11)
             | layer:pool-max                                                   (12)
<pad> ::= padding:same                                                          (13)
             | padding:valid                                                    (14)
<classification> ::= <fully-connected>                                          (15)
<fully-connected> ::= layer:fc <activation> [num-units,int,1,128,2048 <bias>    (16)
<activation> ::= act:linear                                                     (17)
             | act:relu                                                         (18)
             | act:sigmoid                                                      (19)
<bias> ::= bias:True                                                            (20)
             | bias:False                                                       (21)
<softmax> ::= layer:fc act:softmax num-units:10 bias:True                       (22)
<learning> ::= learning:gradient-descent [lr,float,1,0.0001,0.1]                (23)
```

**Fig. 1** Example grammar for the encoding of CNNs

grammar that encodes the parameters of CNNs is shown in Fig. 1. The name of the symbols in the GA rule must match one and only one of the production-rules of the BNF grammar. It is now clear that the features of the GA rule is either a convolution or a pooling layer (first two lines of Fig. 1). The pooling layer has 4 parameters: pool type, kernel size, stride, and padding (line 10). The pool type and padding have closed values (e.g., the pool type is average or maximum) (lines 11 and 12), and the remaining parameters are real-valued; when dealing with real parameters we use a 5-tuple: variable name, variable type (i.e., float or int), number of values to generate, minimum and maximum values.

The novelty of DENSER relies on the combination of these two levels. Without the GA level it would not be possible to encapsulate the genetic material, which facilitates the application of the genetic operators, and thus eases evolution. Without the DSGE level it would be unfeasible to make DENSER a general purpose approach: we only need to adapt the grammar to apply DENSER to the evolution of different network types, pertaining different layers, or to solve

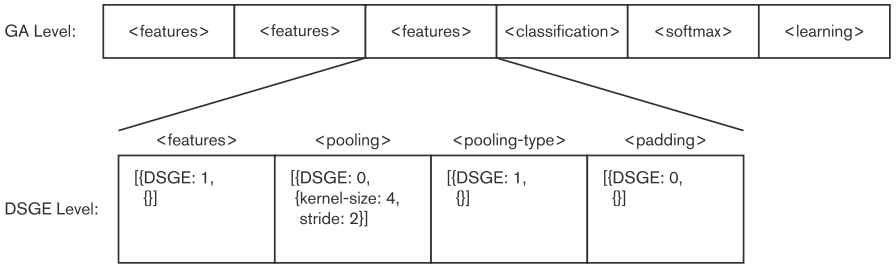


**Fig. 2** Example of the genotype of a candidate solution that encodes a CNN

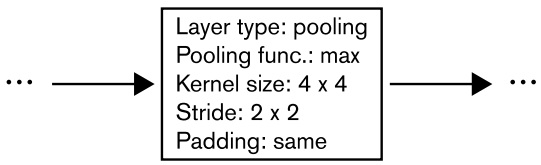


**Fig. 3** Phenotype corresponding to the layer specified in Fig. 2

different tasks; the grammatical representation also turns out to make the incorporation of domain specific knowledge straightforward.

Figure 2 depicts an example of the genotype of a candidate solution, based on the grammar of Fig. 1 and on the example GA level structure introduced above. The figure depicts the complete GA level of a candidate solution, and an overview of the DSGE level of a specific layer. Figure 3 presents the phenotype corresponding to the layer which has the DSGE genotype detailed. To better understand the mapping from the genotype to the phenotype refer to Sect. 5.3.

The initial population is generated at random, i.e., we generate a number of layers within the allowed range (respecting the defined GA structure). Then, for each layer we set their parameters stochastically.

### 5.2 Genetic operators

To promote the evolution of the candidate solutions we rely on crossover and mutation operators specifically designed for the manipulation of ANNs. These operators act on the two levels of the genotype.

#### 5.2.1 Crossover

One of the advantages of having two genotypic levels is that the GA level encodes each evolutionary unit (layer, learning, or data augmentation) separately. Since the genetic material is encapsulated, devising efficient crossover operators is facilitated. We develop two operators, which are probabilistically applied. In the context of this paper, the term module refers to a set of layers that belong to the same index of the GA level genotype structure. For example, in the above example the symbol features has between 1 and 10 layers; therefore the feature's module is composed by between 1 and 10 layers (all those that have

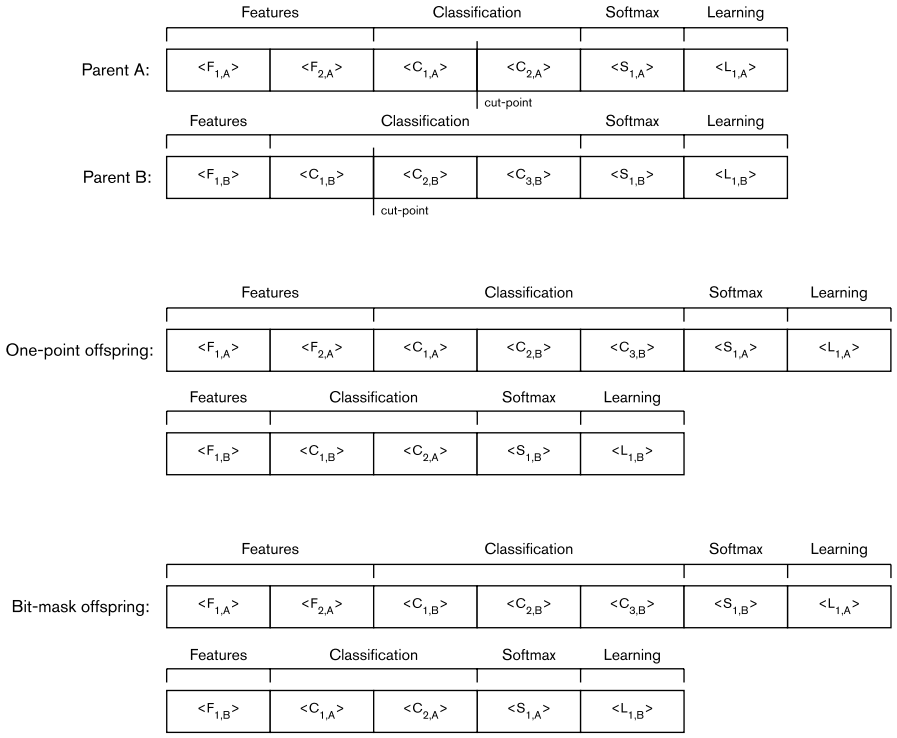


**Fig. 4** Example of the two crossover operators. The example focuses on the GA level of the genotype. For the bit-mask crossover the mask is 1001, which is associated to the features, classification, softmax and learning modules, respectively

their derivation starting with the same symbol features). In particular, in Fig. 2, the feature's module is composed by the first 3 layers.

One of the crossovers changes layers within the same module. To do so, we first select a given module from the two parents (selected by tournament), and then we apply a one-point crossover to the module, without changing any of the remaining modules. Note that the parents can have a distinct number of layers; to deal with that the cutting point is randomly generated considering the individual that has the least number of layers in the module.

The other crossover operator is loosely based on the uniform operator for binary representations, and acts upon the modules swapping them between individuals. Whilst the previous operator acts inside a given module this changes entire modules between individuals, i.e., not only one layer is changed, but rather a set of layers. To a certain extent, the first crossover can be seen as performing exploitation, while the later is more targeted at exploration.

An example of the application of the two crossover operators is shown in Fig. 4.

#### 5.2.2 Mutation

We design specific operators for each of the genotypic levels. The mutations that act upon the GA level aim at manipulating the structure (in terms of evolutionary units[3]):

**Add unit** A new evolutionary unit is generated at random with the initial symbol for the grammatical derivation being the one of the module where the layer will be placed. This operator can only be applied in modules where the maximum number of layers has not been yet reached;

**Replicate unit** Similar to the previous mutation operator, but instead of generating a new random unit uses one that is already in the genotype and copies it into another random valid position of the module. The copy is done by reference, meaning that if at any given time the layer or some of its parameters are changed, the modifications are propagated to the copies;

**Remove unit** Deletes a random unit from a given module. It is only possible to remove an evolutionary unit if, after removal, the number of units of the module it belongs to is still above the minimum threshold.

The previous operators act only at a macro level, and thus do not change the parameters of the layers. This is accomplished at the DSGE level:

**Grammatical mutation** As in standard DSGE, an expansion possibility is replaced by another valid one;

**Integer mutation** An integer block is replaced by a new one, where the integers are generated at random, within the allowed range;

**Float mutation** Similar to the integer mutation, but where instead of randomly generating new values, a Gaussian perturbation is applied.

### 5.3 Evaluation

To evaluate each candidate solution, i.e., each DNN, we have to perform 4 sequential steps: (i) map the genotype to the phenotype; (ii) map the phenotype into a trainable model; (iii) train the DNN model; (iv) assess the performance of the model, which will determine the fitness of the candidate solution. To facilitate the evaluation we use Keras [11]: an API with GPU support that aids in the train of ANNs; Keras runs on top

[3] Recall that in the context of this work a evolutionary unit can be rather a layer, or any set of arguments related with for example learning or data augmentation.

of TensorFlow [1]. The GPU support is vital given the need to train every candidate solution, and the time and cost associated to it.

To decode the genotype, the GA level of each candidate solution is traversed linearly. For each position of the GA level we decode the corresponding DSGE level (see Sect. 4). The difference to the standard DSGE relies on the expansion of the real values; in the standard DSGE real values are encoded using a production rule for floating point numbers, and usually each expansion encodes a number between 0 and 9; we encode the real values directly, as in GAs; each real-value is used only once too, as if we were dealing with the expansion of any other terminal symbol.

Independently of the dataset, to use DENSER, and in order to report unbiased results, we need three disjoint data partitions:

| | |
|---|---|
| Train | —To train the network using the defined or evolved learning parameters. The parameters vary depending on the used learning algorithm; |
| Validation | —To evaluate the performance of the network during evolution; |
| Test | —Kept aside from the evolutionary process, and used to evaluate the performance of the best models on unseen data, so we can assess the generalisation ability of the evolved networks. |

Each network is trained with Keras during 10 epochs, and the fitness is the best performance on the validation set on the 10 epochs. Currently we are using the accuracy as the evaluation metric; we decided for this metric to enable comparison with other works, and because we will not be dealing with unbalanced datasets; in any case, changing the accuracy to any other metric (e.g., f-measure) is straightforward. Longer trains, i.e., more epochs, would enable higher validation performances, and a better grasp of the training behaviour. On the other hand, longer trains would slow the evolutionary process, as the evaluation stage would be more time consuming. Data augmentation is used, namely, padding, horizontal flips, and random crops. For more insights on the followed data augmentation approach refer to [59].

## 6 Experimentation

To evaluate DENSER we conduct experiments on the automatic search of CNNs for the classification of the CIFAR-10 dataset. To assess the generalisation, robustness, and scalability of the DNNs discovered by DENSER we investigate the performance of the CNNs that perform best on CIFAR-10 on a wide set of object recognition benchmarks: MNIST, Fashion-MNIST, SVHN, Rectangles, and CIFAR-100. The benchmarks are further detailed in Sect. 6.1. The experimental setup is described in Sect. 6.2, and the analysis of the results is carried out in Sects. 6.3 and 6.4. The best trained models have been released at http://github.com/fillassuncao/denser-models.

### 6.1 Description of the datasets

The CIFAR-10 [32] dataset is highly used by state-of-the art methods, allowing a comparison between the DNNs found by DENSER and those that are designed by other evolutionary and non-evolutionary approaches. Furthermore, it is a moderately complex benchmark composed by $32 \times 32$ RGB real-world images; the dataset is large enough to allow an accurate understanding of the time it takes to find high performing DNNs. For the above reasons we decided to use the CIFAR-10 to evaluate the effectiveness of DENSER on automatically discovering DNNs.

To test the generalisation and robustness of the best models generated by DENSER we use MNIST [33]. The MNIST dataset is composed of $28 \times 28$ grayscale handwritten digits between 0 to 9. To better investigate the robustness of the evolved DNNs in addition to the standard MNIST we also test several variations,[4] namely:

MNIST Rotated —the digits are rotated between 0 and 360°;
MNIST Background —instead of a clean white background, a real-world image is used as background for the digit;
MNIST Rotated + Background —combines the previous two modifications, i.e., the digits are rotated, and an image is used in the background.

The standard MNIST is widely used as a dataset to establish comparisons between approaches. Nonetheless, it is known to be an easy task, where simple CNNs can achieve accuracies close to 99% [51]. For that reason, we have decided to also test on Fashion-MNIST [65] and SVHN [43]. Fashion-MNIST has the same format as MNIST (i.e., $28 \times 28$ grayscale images), but where the digits are replaced by clothing items; SVHN has an objective similar to MNIST (i.e., identify digits between 0 and 9), but where the images are from Google Street View, and thus are significantly harder to classify.

The scalability of the models evolved by DENSER is tested on Rectangles,[5] and CIFAR-100 [32]. In the Rectangles dataset the goal is to distinguish between tall and wide rectangles placed in 28×28 images; two setups are tested: (i) without any background image; (ii) with a background image (similar to MNIST Background Image). The CIFAR-100 is composed by the same images as CIFAR-10, but where they are to be separated into 100 disjoint classes.

Table 1 details numeric properties of the used datasets, namely, the number of train and test instances, the input size, and the number of classes. As mentioned previously, the evolutionary search for CNNs is carried out using the CIFAR-10 dataset. For that reason, in this case, we partition the dataset into 3 disjoint sets: train,

[4] For more information about the MNIST variants check http://www.iro.umontreal.ca/~lisa/twiki/bin/view.cgi/Public/MnistVariations.

[5] More information regarding the Rectangles dataset can be found in http://www.iro.umontreal.ca/~lisa/twiki/bin/view.cgi/Public/RectanglesData.

**Table 1** Numerical overview of the used datasets

| Daset | #Train | #Test | Input size | #Classes |
|---|---|---|---|---|
| MNIST [33] | 60,000 | 10,000 | 784 | 10 |
| MNIST variants | 50,000 | 12,000 | 784 | 10 |
| Fashion-MNIST [65] | 60,000 | 10,000 | 784 | 10 |
| SVHN [43] | 73,257 | 26,032 | 3072 | 10 |
| Rectangles | 12,000 | 50,000 | 784 | 2 |
| CIFAR-10 [32] | 50,000 | 10,000 | 3072 | 10 |
| CIFAR-100 [32] | 50,000 | 10,000 | 3072 | 100 |

**Fig. 5** From top to bottom, each row depicts randomly selected instances of the MNIST, MNIST Rotated, MNIST Background, MNIST Rotated Background, Fashion-MNIST, SVHN, Rectangles, Rectangles Background, and CIFAR, respectively. Each instance represents one of the classes of the dataset; the Rectangles dataset only has 2 classes, the first five belong to one of the classes, and the last five to the other

validation and test. In all the other cases, since we use the DNNs found for CIFAR-10, only the train and test sets are needed. Figure 5 depicts example instances of each of the datasets.

**Table 2** Experimental parameters

| | Value |
|---|---|
| Evolutionary engine parameter | |
| Number of runs | 10 |
| Number of generations | 100 |
| Population size | 100 |
| Crossover rate | 70% |
| Mutation rate | 30% |
| Tournament size | 3 |
| Elite size | 1% |
| Dataset parameter | |
| Train set | 42,500 instances |
| Validation set | 7500 instances |
| Test set | 10,000 instances |
| Train parameter | |
| Number of epochs | 10 |
| Loss | Categorical Cross-entropy |
| Batch size | 125 |
| Learning rate | 0.01 |
| Momentum | 0.9 |
| Data augmentation parameter | |
| Padding | 4 |
| Random crop | 4 |
| Horizontal flipping | 50% |

### 6.2 Experimental setup

Table 2 details the experimental parameters used for conducting the evolutionary runs on the automatic discovery of DNNs able to effectively classify the CIFAR-10. The parameters are divided into 4 categories: (i) evolutionary engine—associated with the evolutionary algorithm; (ii) dataset—partitioning of the CIFAR-10; recall that the test set is kept out of the evolutionary cycle; (iii) train—backpropagation algorithm parameters; (iv) data augmentation—used to generate more data, and prevent overfitting.

To encode CNNs we use the grammar of Fig. 6, and the following GA structure: [(features, 1, 30), (classification, 1, 10), (softmax, 1, 1)]. Our search space encompasses DNNs with up to 40 hidden-layers: at most 30 convolution or pooling layers followed by up to 10 fully-connected layers. In this set of experiments we decided not to optimise the learning parameters; that is why there is no production rule associated with learning (the parameters of Table 2, section train parameter are used).

The longer the DNNs are trained the better grasp we have regarding their long term behaviour. However, the train of DNNs is a computationally expensive task, mostly due to the burden of the evolutionary process, since each network needs to to be evaluated for its fitness. Likewise, similarly to the works of Miikkulainen et al. [38], and Suganuma et al. [59] we perform short trains of each DNN; in particular,

```
<features> ::= <convolution>
             | <pooling>
<convolution> ::= layer:conv [num-filters,int,1,32,256] [filter-shape,int,1,1,5]
                  [stride,int,1,1,3] <padding> <activation> <bias>
                  <batch-normalisation> <merge-input>
<batch-normalisation> ::= batch-normalisation:True
                        | batch-normalisation:False
<merge-input> ::= merge-input:True
                | merge-input:False
<pooling> ::= <pool-type> [kernel-size,int,1,1,5] [stride,int,1,1,3] <padding>
<pool-type> ::= layer:pool-avg
              | layer:pool-max
<padding> ::= padding:same
            | padding:valid
<classification> ::= <fully-connected>
<fully-connected> ::= layer:fc <activation> [num-units,int,1,128,2048 <bias>
<activation> ::= act:linear
               | act:relu
               | act:sigmoid
<bias> ::= bias:True
         | bias:False
<softmax> ::= layer:fc act:softmax num-units:10 bias:True
```

**Fig. 6** Grammar used for the evolutionary runs on the evolution of CNNs for the CIFAR-10

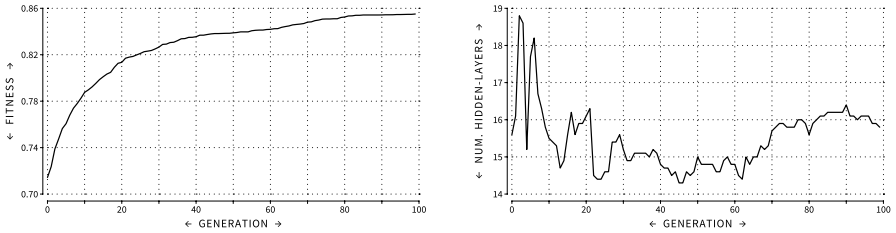


**Fig. 7** Evolution of the fitness (left) and number of hidden-layers (right) of the best individuals across generations. Results are averages of 10 independent runs

we use 10 epochs. After the evolutionary runs, we take the fittest network and perform longer trains, with 400 epochs, and the same train policy. For these longer trains the train and validation sets are merged, and the performance measured on the test set. For the same reason, only 10 evolutionary runs are conducted.

During evolution invalid solutions can be generated. This occurs because some of the layers change the input shape; more specifically, pooling layers downsample the input space; the same can happen with convolution layers, if padding is not used.

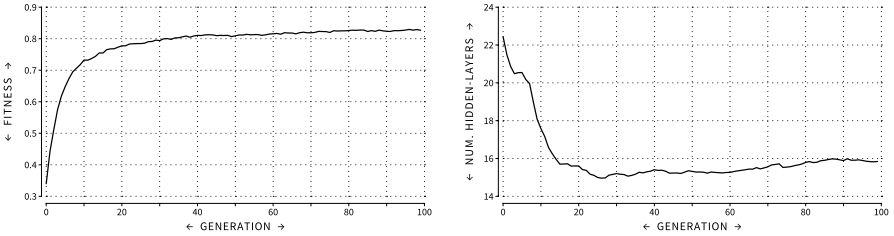


**Fig. 8** Evolution of the fitness (left) and number of hidden-layers (right) of the overall population across generations. Results are averages of 10 independent runs

To overcome invalid solutions, the layers that generate invalid shapes[6] are not considered to build the model that is later trained and evaluated, i.e., these layers are non-coding genotype. We do not repair the genotype because to fix a layer we would have to change its parameters, and the layer can be a copy by reference, and thus we would possibly be changing too much of the genotype.

### 6.3 Evolution of CNNs for the CIFAR-10

We start by analysing the ability of DENSER to search for and generate DNNs able to solve a classification task. We focus on object recognition, as an active research area in DL, and thus we target the automatic evolution of CNNs for the CIFAR-10 dataset.

The evolution of the average fitness (i.e., classification accuracy) and number of hidden-layers of the fittest CNNs across generations is depicted in Fig. 7. A brief perusal of the fitness evolution indicates that DENSER is working properly, as solutions tend to be fitter as time passes; convergence happens around the 80th generation, with a classification accuracy of approximately 85%. The behaviour of the number of hidden-layers is more erratic, and two different and contradictory patterns are observable. From the start of evolution and until approximately the 60th generation an increase in performance is accompanied by a decrease in the number of hidden-layers; this changes from the 60th generation until the last generation where an increase in performance is followed by an increase in the number of hidden-layers.

To support the previous result we compute the correlation between the average fitness values of the best individuals and the average number of hidden-layers, per generation. The Pearson correlation reports a coefficient of −0.7166 (moderate negative correlation) for the correlation between the two metrics before the 60th generation; after the 60th generation the coefficient is 0.9204 (strong positive correlation). This result is explained after the fact that in the first generation the randomly generated solutions have a large number of hidden-layers (approximately 15.6), which correspond to very deep networks. However, since the numeric parameters of each

[6] An invalid shape in this scenario occurs when the downsampling generates a negative size for the shape of the output of the layer.

layer are set at random, they hardly provide any meaningful parameterisation. As evolution proceeds and optimises the numeric values, the best solutions can steadily increase the number of hidden-layers to improve performance.

The previous explanation suggests that it may be advantageous to start evolution from shallower networks, i.e., constrain the initialisation procedure to a maximum number of hidden-layers. Although this is common practice in many approaches, we strive to provide the least amount of knowledge and bias to the system. Therefore, it is remarkable that starting from very deep networks, DENSER finds out that trimming is necessary in the first generations.

In addition to analysing the best evolved solutions we also inspect the overall quality of the population. Figure 8 depicts the evolution of the fitness, and the number of hidden-layers, across generations at the population level. The conclusions are in line with those reported for the analysis of the best solutions, however the change in behaviour occurs earlier (around the 25th generation). Before the 25th generation the Pearson correlation between the fitness and number of hidden-layers reports a coefficient of $-0.89$ (strong negative correlation), and a coefficient of 0.8801 after the 25th generation (strong positive correlation). The change in behaviour happens earlier than when considering only the best solutions because in the first generations the population has many low performing solutions that are quickly discarded.

By the end of evolution the best solutions and the overall population have very similar fitness values of approximately 85% and 83%, respectively; the same happens with the number of hidden-layers, which is close to 16 in both analysis. This reinforces the idea that DENSER is working properly, and that the population is converging towards the best solutions.

The fittest network found during evolution is represented in Fig. 9. The network is selected based on the accuracy on the validation set.[7] When merging the output of the convolution layers with the input the shape of the signals must be the same; if the number of channels is not equal we pad the smallest of the signals; if the dimensionality of the signals is different we down-sample the largest one using max pooling. Between the merge and its inputs the pooling and padding operations necessary to make the merge valid are omitted.

The most puzzling characteristic of the evolved networks is the importance of the fully-connected layers that are used at the end of the topology. Other approaches on the evolution of CNNs tend to disregard fully-connected layers, and focus only on convolution and pooling layers. We tried to remove some of the fully-connected layers, and preliminary results show that the performance of the networks degenerated. Moreover, to the best of our knowledge, the use of fully-connected layers is not usual, and it is fair to say that a human would never think of such topology, which makes this evolutionary outcome remarkable.

[7] It is later shown that this network reports as one of the highest performing in terms of test accuracy, and thus it generalises well.

**Fig. 9** Topology of the fittest network found during evolution. The current network has 5 layer types: convolution (Conv), max. pooling (MaxPool), fully-connected (FC), merge, and activation. The layer blocks have the following format: Conv:num-filters:filter-shape:stride:padding:batch-normalisation:bias; MaxPool:kernel-size:stride:padding; FC:num-units:bias; Activation:type. For more information about the padding type refer to the Keras documentation; when batch-normalization or bias are not used we set the parameter to none

#### 6.3.1 Further training

Once the evolutionary process is complete, the fittest network found in each run, i.e., the ones obtaining the highest accuracy value on the validation set, are re-trained 5 times. Trains are conducted multiple times because the backpropagation algorithm is stochastic (due to the different weight initialisations), and thus different trains can provide significantly different performances. The accuracy results are averaged over 5 trains. From this point onward (including upcoming sections) the classification accuracy results concern the performance obtained on the test set.

First, we train the networks with the same learning rate policy that is used in evolution (see Table 2), but during 400 epochs instead of 10. With this setup we obtain, on average, a classification accuracy of 88.41%. We also experiment different test policies, namely the one described by Snoek et al. in [54]: for each instance of the test set we generate 100 augmented images; the label assigned by the model to the instance is the class that gives the maximum average confidence value on the 100 augmented images. With this methodology the average classification accuracy of the fittest networks increases to 89.93%.

Notwithstanding the increase in performance by using the methodology followed by Snoek et al., the results still seem far from the state-of-the-art. This happens because the results of the state-of-the-art only focus on the performance reported by a single DNN. The classification accuracy of the highest performing CNN found by DENSER (using the methodology by Snoek et al.) is of 92.70%. This result despite competitive with the state-of-the-art, does not surpass it. To avoid a biased choice of the best network, we base the selection on the network that attains the highest train accuracy; this happens to be also the network that provides the highest test accuracy, which proves that the networks generalise well.

To investigate if it is possible to increase the performance of the fittest networks we re-train them with a different learning rate policy; in particular, we follow the methodology of CGP-CNN [59]: a varying learning rate that starts with 0.01; on the 5th epoch is increased to 0.1; by the 250th epoch is decreased to 0.01; and finally at the 375th epoch reduced to 0.001. With this train policy, the mean accuracy of all the best networks increases from 88.41% to 92.51%, without using data augmentation on the test set; and to 93.29% with data augmentation on the test set. The previous results are the outcome of multiple networks; if considering only the network that reports the highest error on the train set, the best test classification accuracy is of 93.38% without data augmentation; and of 94.13% with data augmentation. This last result is highly competitive, and is aligned with the state-of-the-art on approaches that automatically search for the best DNN for classifying the CIFAR-10.

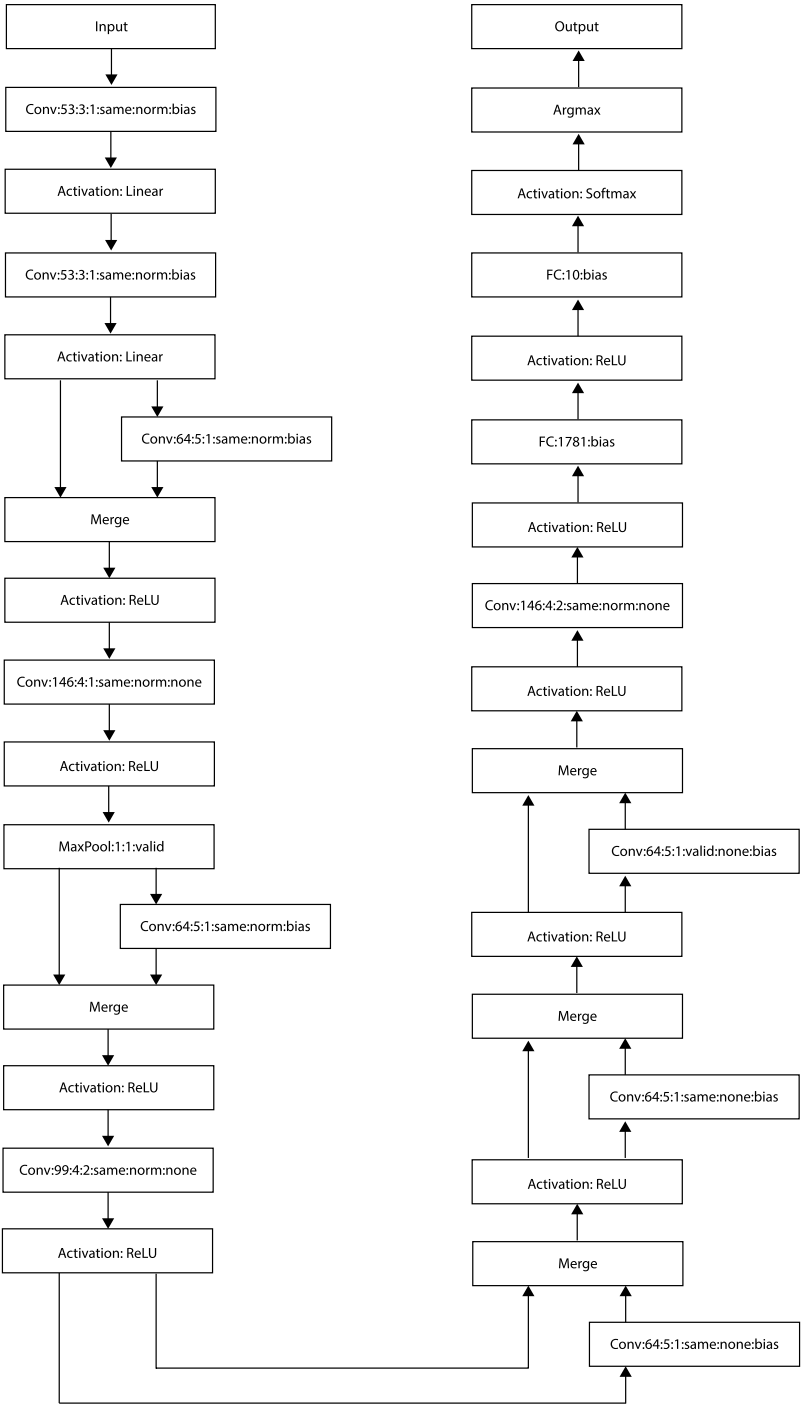
Input
Conv:53:3:1:same:norm:bias
Activation: Linear
Conv:53:3:1:same:norm:bias
Activation: Linear
Conv:64:5:1:same:norm:bias
Merge
Activation: ReLU
Conv:146:4:1:same:norm:none
Activation: ReLU
MaxPool:1:1:valid
Conv:64:5:1:same:norm:bias
Merge
Activation: ReLU
Conv:99:4:2:same:norm:none
Activation: ReLU
Output
Argmax
Activation: Softmax
FC:10:bias
Activation: ReLU
FC:1781:bias
Activation: ReLU
Conv:146:4:2:same:norm:none
Activation: ReLU
Merge
Conv:64:5:1:valid:none:bias
Activation: ReLU
Merge
Conv:64:5:1:same:none:bias
Activation: ReLU
Merge
Conv:64:5:1:same:none:bias

**Table 3** Overview of the results reported by DENSER on the CIFAR-10

| Networks | Test data augmentation | Epochs | LR Policy-1 (%) | LR Policy-2 (%) |
|---|---|---|---|---|
| AVG best | No | 10 | 85.51 | – |
| AVG best | No | 400 | 88.41 | 92.51 |
| AVG best | Yes | 400 | 89.93 | 93.29 |
| Best | No | 400 | 91.75 | 93.38 |
| Best | Yes | 400 | 92.70 | 94.13 |

The experimental results indicate that it is possible to obtain competitive results using evolutionary approaches and that it is possible to do so with limited computational resources, using a low number of train epochs (10) during evolution. Table 3 summarises the results obtained so far with the CIFAR-10; the learning rate (LR) Policy-1 refers to the train policy used during evolution (with 400 epochs); the LR Policy-2 refers to the variable methodology followed by CGP-CNN.

#### 6.3.2 Ensembling

Previous research indicates that the best results are obtained when the varying learning rate policy from Suganuma et al. [59], and when dataset augmentation is applied to the test set. For each network we performed 5 independent trains, where the initial conditions are different (due to the initial weights). Another source of stochastic behaviour is data augmentation. Therefore, despite having the same topology the networks may have slightly different behaviours. For that reason we investigate if an ensemble composed of the 5 different trains performs better than the individual trained networks. The ensemble decision is taken as when applying data augmentation to the test set, i.e., we average the confidence values reported by all voters; and the decision is the class that reports a maximum average confidence value. The ensemble formed by the 5 trains of the best performing network achieves an accuracy of 95.14% (superior to the previous 94.13%).

In addition to testing ensembles formed from different trains of the same network we also investigate the performance of ensembles formed by different networks. The decision of the ensemble is taken as before. In particular, we ensemble the two fittest networks found by DENSER, and for each of them we consider the 5 independent trains, i.e., the ensemble is formed by 10 voters. This setup obtains an accuracy of 95.22%, which is slightly superior to the one obtained when considering only the fittest network.

### 6.4 Generalisation, robustness, and scalability

To better understand the quality of the DNNs generated by DENSER we investigate their: (i) generalisation—ability to solve multiple tasks; (ii) robustness—resilience to small modifications of the original input; and (iii) scalability—capacity to solve problems with less and more classes. To that end, we take the fittest DNNs

and apply them in the classification of other benchmarks, which were not used for finding the topology. In particular, we test on MNIST and its variants, on Fashion-MNIST, SVHN, Rectangles, and CIFAR-100.

#### 6.4.1 MNIST and variants

To investigate the robustness of the highest performing DNN discovered by DENSER we re-train the network on the MNIST dataset and apply it to the classification of the MNIST variants: MNIST Rotated, MNIST Background, and MNIST Rotated + Background. In this second step, the networks are not re-trained, i.e., the weights tuned for the standard MNIST are directly used. We train the network with LR Policy-2, the learning rate policy that obtained the highest results. Trained on standard MNIST, the network reports an average classification accuracy of 99.65%, without data augmentation over the test set, and an average classification accuracy of 99.70%, using data augmentation.

For the MNIST Rotated, MNIST Background, and MNIST Rotated + Background, using the previously trained DNN the classification accuracy is of 46.63%, 42.48%, and 22.27%, respectively. These results were obtained without applying data augmentation to the test set instances, because the images that are feed to the network are already "augmented" versions of MNIST. Compared to the 99.65% performance on MNIST the results may be considered under-performing. Looking at the first 4 rows of Fig. 5 it is clear that the dataset instances of the MNIST dataset (first row), and its variants (second to fourth rows) are very different; in addition, our data augmentation method does not consider image rotations.

However, when we re-train the network for the MNIST variants we obtain average accuracies of 97.71%, 98.62%, and 92.66%, for the MNIST Rotated, MNIST Background, and MNIST Rotated + Background, respectively. This shows that the evolved DNN topology is perfectly able to cope with these variants of MNIST. Thus, the poor performance reported earlier is explained by the lack of adequate examples in the standard MNIST dataset, even when augmented, to allow the network to generalise to these new circumstances.

Finally, we re-train the network using instances from all MNIST variants, i.e., instead of training the network 4 times, one with each of the variants plus the standard dataset, we do a single train, with 15,000 instances from the standard dataset and from each one of the variants. The train instances are randomly selected, in a stratified way, meaning that the dataset is balanced. This setup yields an average classification accuracy of 95.67%.

#### 6.4.2 Fashion-MNIST and SVHN

MNIST despite important as a baseline does not serve as a stress test, because nowadays it is considered an easy to solve problem, where a simple MLP is able to achieve performances close to 90%. That is the reason why we look into the performance in other more challenging datasets to better assess the generalisation ability of the DNNs generated by DENSER.

We take the best performing network and re-train it on the Fashion-MNIST, and SVHN datasets, with LR Policy-2. On the Fashion-MNIST we get average classification accuracies of 94.23% (with no data augmentation on the test set), and 94.70% (with data augmentation). With the SVHN dataset we get average classification accuracies of 95.44% (with no data augmentation on the test set), and 96.23% (with data augmentation).

As in previous experiments, we test ensembling the different trains of a single model, or of the two fittest models. On the Fashion-MNIST, the ensembles report classification accuracies of 95.11%, and 95.26%, considering only the fittest, or the two fittest models respectively. The same analysis on SVHN reports classification accuracies of 96.88%, and 97.02%, considering only the fittest, or the two fittest models respectively.

The results on MNIST, Fashion-MNIST, and SVHN allow us to state that the models evolved by DENSER generalise, i.e., despite not directly evolved for solving these tasks, the DNNs perform well beyond evolution. The comparison with the state-of-the-art is carried out in Sect. 6.5.

#### 6.4.3 Rectangles and CIFAR-100

In all of the above experiments we have always sought to solve problems with 10 classes. To analyse the scalability of the models generated by DENSER we test with Rectangles, an artificial problem with 2 classes, and CIFAR-100, with 100 classes. We test in the two Rectangles variants: with and without background images. The same flow-chart is followed: we take the fittest networks and re-train them on the Rectangles variants and CIFAR-100 datasets.

The fittest networks excel in the classification of Rectangles: without background we reach an average test accuracy of 100%; with background images we obtain a classification accuracy of 99.64%. Data augmentation on the test set does not improve the results. We do not experiment ensembling with the models trained in the Rectangles without background as no improvements are possible; ensembling the classifiers trained with background slightly increases the classification accuracy to 99.73%.

On the CIFAR-100, we report classification accuracies of 73.33%, and 74.94%, not performing and performing data augmentation over the test set, respectively. The ensemble formed only by the trains of the fittest model gives an average accuracy of 77.51%; with the two fittest models the performance increases to 78.75%.

### 6.5 Discussion

In this section we make overall remarks of all the conducted experiments, and compare the obtained results with the state-of-the-art. Recall that we performed two independent experiments: (i) evolutionary search of CNNs for the classification of the CIFAR-10; and (ii) analysis of the generalisation, robustness, and scalability of the networks generated by DENSER for the classification of the CIFAR-10; to

**Table 4** Performance of different CNNs on the classification of the datasets used throughout the paper

| Approach | Dataset | Accuracy (%) |
|---|---|---|
| ResNet [25] | MNIST | 97.9 |
| MetaQNN [6]* | | 99.56 |
| Simard et al. [51] | | 99.60 |
| Graham [20] | | 99.68 |
| VGG [52] | | 99.68 |
| **DENSER*** (best network) | | 99.70 |
| VGG [52] | Fashion-MNIST | 93.50 |
| **DENSER*** (best network) | | 94.70 |
| ResNet [25] | | 94.90 |
| **DENSER*** (ensemble) | | 95.26 |
| Sermanet et al. [50] | SVHN | 95.10 |
| **DENSER*** (best network) | | 96.23 |
| **DENSER*** (ensemble) | | 97.02 |
| MetaQNN [6]* | | 97.72 |
| Goodfellow et al. [19] | | 97.84 |
| Loshchilov and Hutter [35]* | CIFAR-10 | 90.70 |
| VGG [52] | | 92.26 |
| CoDeepNEAT [38]* | | 92.70 |
| MetaQNN [6]* | | 93.08 |
| ResNet [25] | | 93.39 |
| Snoek et al. [54]* | | 93.63 |
| CGP-CNN [59]* | | 94.02 |
| **DENSER*** (best network) | | 94.13 |
| **DENSER*** (ensemble) | | 95.22 |
| Real et al. [47]* | | 95.60 |
| Graham [20] | | 96.53 |
| ResNet [25] | CIFAR-100 | 71.14 |
| VGG [52] | | 71.95 |
| Snoek et al. [54]* | | 72.60 |
| MetaQNN [6]* | | 72.86 |
| Graham [20] | | 73.61 |
| **DENSER*** (best network) | | 74.94 |
| Real et al. [47]* | | 77.00 |
| **DENSER*** (ensemble) | | 78.75 |

The AlexNet, RestNet, and GoogleNet results for the MNIST and Fashion-MNIST are from https://github.com/zalandoresearch/fashion-mnist

Bold indicates the results reported by DENSER. The results are sorted in ascending order

Automatic approaches are marked with an *

measure these properties we use the MNIST, Fashion-MNIST, SVHN, Rectangles, and CIFAR-100 datasets.

Evolution works as expected: DENSER promotes the evolution of high performing networks, and at the end of evolution the average fitness of the population is very close to the average fitness of the best candidate solutions. In addition, the evolutionary method replicates behaviours that are common in hand-designed networks; in particular, when at the beginning of evolution the DNNs are trimmed to facilitate the optimisation of the parameters. On the other hand, DENSER generates DNNs that are novel, and that human-designers would unlikely think of.

The networks that report the best classification accuracy are applied to a wide range of datasets, and for all of them they report high accuracy values, as such, DENSER is able to evolve DNNs that generalise well. In particular, we analyse the robustness by predicting the class labels of MNIST perturbed instances (rotated and/or added background) using CNNs that are trained on the standard MNIST; at first it may seem that the networks underperform, but we attribute that to the used data augmentation method that does not promote the rotation of the instances (the MNIST variants where the performance is the lowest); if we retrain the CNNs with images from all variants the CNNs attain performances that are close to the ones obtained on standard MNIST. Finally, the scalability of the evolved CNNs is tested on the Rectangles and CIFAR-100; the networks excel, with the performance on the CIFAR-100 surpassing the state-of-the-art results (discussed next).

Table 4 enumerates state-of-the-art results on the classification of the multiple datasets considered in the current paper. We focus on the comparison of our results with those reported by other CNNs. Some of the state of the art results were obtained by hand-designed networks, instead of using an automatic approach (evolutionary or any other form of automatisation). As such, the ones that are obtained by automatic CNN designing methods are marked with an *.

We only consider for comparison with the state-of-the-art the datasets that are most often used. For the MNIST, Fashion-MNIST, SVHN, CIFAR-10, and CIFAR-100 datasets we achieve average test classification accuracies of 99.70%, 94.70%, 96.23%, 94.13%, and 74.94%, respectively. The previous results are not considering the ensembles, which slightly increase the classification accuracy. Comparing to the results of Table 4 we can observe that the results reported by DENSER are superior to the state-of-the-art on the MNIST, and Fashion-MNIST, and are competitive in the remaining benchmarks.

Focusing on the analysis of the CIFAR-10, and CIFAR-100, DENSER obtains results that are 1.47% and 2.06% below the methodology of Real et al. [47], the best performing automatic approach for these benchmarks. We note that Real et al. use ensembling, and thus their results are to be compared with our ensembling ones. DENSER reports average ensembling test accuracies of 95.22%, and 78.75%, respectively for the CIFAR-10, and CIFAR-100. These results show that on CIFAR-10 Real et al. results are slightly superior (95.6% vs 95.22%); on CIFAR-100 the results reported by DENSER are superior (78.75% vs 77%). A deeper analysis of their work shows that their search space is considerably smaller than ours, with several constraints on the convolution parameters, and no pooling or fully-connected layers. DENSER searches a larger space obtaining a similar performance on the CIFAR-10. For CIFAR-100, Real et al. re-conduct evolution from scratch; we just

take the fittest network found for CIFAR-10 and apply it to the CIFAR-100, outperforming Real et al.'s results. This is because the search space of DENSER is capable of generating DNNs that Real et al. cannot, because the search space appears to have been optimised for CIFAR-10.

A comparison with hand-designed networks is also possible from the analysis of Table 4, with all the results not obtained by automatic approaches (i.e., all of those not marked with a *) the outcome of iterative trial-and-error human design. Concerning only the best generated model, DENSER outperforms the human-designed CNNs in the MNIST, and CIFAR-100 datasets, and obtains competitive results in the Fashion-MNIST. The results obtained on SVHN, and CIFAR-10 are below the ones reported by human-designed models; in particular, in CIFAR-10, Graham [20] obtain an accuracy that is 2.4% above the fittest DENSER model, but for that they propose fractional max pooling, which is not included in the search space of DENSER (but that can be added in future experiments).

Regarding the architectures of the hand-designed models they tend to be different than what the evolutionary process generates. When the networks are designed by a human we often find blocks of layers that are replicated several times, e.g. the convolution layer followed by pooling [19], or a set of convolutional layers with different resolution levels [52]. As it is observable from the model of Fig. 9 this will very hardly happen when evolving the topology from scratch, without defining what the blocks are, i.e., with no prior knowledge about the search space. The same happens if we compare the models designed by other evolutionary systems with those designed by human practitioner; unless we specifically define a macro structure that inputs the notion of block [59], the generated networks will have a novel architecture, different from the vast majority of the existent models. From our point of view this is a key advantage of applying EC to search effective architectures: it enables us to obtain models that a human designer would unlikely think of. The networks generated by DENSER and other evolutionary systems are similar; this does not mean that the same sequence of layers are used, but rather that the build models have a very flexible, and unconstrained structure.

## 7 Conclusions and future work

The large amounts of data and the complex tasks that researchers are trying to solve often demand the use of ANNs with deep architectures. The challenge when developing this type of networks relies on the difficulty to find adequate structures and their parameterisation. To tackle this issue, methods that address the automatic generation of ANNs by means of evolutionary methods have been proposed. However, the majority of them are not capable of dealing with the evolution of large scale deep structures, or are too constrained to a specific type of network architecture.

In the current article we describe DENSER: a layer-based NE approach that combines a GA with DSGE. The representation of the candidate solutions in DENSER, i.e., deep networks, is made at two different levels: the GA level, that encodes the ordered linear sequence of layers; and the DSGE level, which encodes the parameters of each single layer. We also design specific genetic operators that focus on the

evolution of ANNs based on the two-level representation of the genotype. The way in which the solutions are encoded allows a two-fold gain: (i) the genetic material is encapsulated, which facilitates the application of the genetic operators; and (ii) the grammatical nature of the method makes it easy to evolve solutions to different problems, or different network structures.

The results show the effectiveness of DENSER. We conduct experiments in the evolution of CNNs for the CIFAR-10, and the results show that DENSER is currently the evolutionary approach, with no prior-knowledge, that generates the highest performing networks. In addition, the evolved networks generalise, are robust, and scale; the most remarkable result of this is the performance on CIFAR-100: without further evolution, when the fittest networks on the CIFAR-10 are re-trained on the CIFAR-100 they obtain performances that surpass the current state-of-the-art results, with an average classification accuracy of 78.75%.

For future work we will study how to improve the optimisation of the learning parameters. The evaluation of the networks is being conducted on just 10 epochs, and thus it is hard to evolve effective learning rate schedules. Therefore, we want to analyse ways to allow longer trains, for example, by performing an iterative sampling of the dataset instances, where the sample is selected based on the regions of the domain the DNN has most difficulties to model [29, 58]. Moreover, we plan on assessing the impact of evolving modular structures, that can allow the resolution of multiple tasks in simultaneous; i.e., each module is responsible for a task, and the DNNs grow as needed, avoiding catastrophic forgetting. The impact of forming ensembles during run-time, instead of considering only the fittest candidate solutions of each run will also be investigated.

**Acknowledgements** This work is partially funded by: Fundação para a Ciência e Tecnologia (FCT), Portugal, under the Grant SFRH/BD/114865/2016. We would also like to thank NVIDIA for providing us Titan X GPUs.

## References

1. M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, Tensorflow: a system for large-scale machine learning. OSDI **16**, 265–283 (2016)
2. F. Ahmadizar, K. Soltanian, F. AkhlaghianTab, I. Tsoulos, Artificial neural network development by means of a novel combination of grammatical evolution and genetic algorithm. Eng. Appl. Artif. Intell. **39**, 1–13 (2015)
3. F. Assunção, N. Lourenço, P. Machado, B. Ribeiro, Evolving the topology of large scale deep neural networks, in *European Conference on Genetic Programming*. Springer, pp. 19–34 (2018)
4. F. Assunção, N. Lourenço, P. Machado, B. Ribeiro, Towards the evolution of multi-layered neural networks: A dynamic structured grammatical evolution approach, in *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO'17*. ACM, New York, NY, USA, pp. 393–400 (2017). https://doi.org/10.1145/3071178.3071286
5. T. Bäck, H.P. Schwefel, An overview of evolutionary algorithms for parameter optimization. Evol. Comput. **1**(1), 1–23 (1993)
6. B. Baker, O. Gupta, N. Naik, R. Raskar, Designing neural network architectures using reinforcement learning. arXiv preprint arXiv:1611.02167 (2016)
7. A. Baldominos, Y. Saez, P. Isasi, Evolutionary design of convolutional neural networks for human activity recognition in sensor-rich environments. Sensors (14248220) **18**(4) (2018)

8. J. Bergstra, Y. Bengio, Random search for hyper-parameter optimization. J. Mach. Learn. Res. **13**, 281–305 (2012)
9. J.S. Bergstra, R. Bardenet, Y. Bengio, B. Kégl, Algorithms for hyper-parameter optimization, in *Advances in Neural Information Processing Systems*, pp. 2546–2554 (2011)
10. C.M. Bishop, *Pattern Recognition and Machine Learning (Information Science and Statistics)* (Springer, Berlin, Heidelberg, 2006)
11. F. Chollet, et al.: Keras. https://keras.io (2015)
12. Z. Chunhong, J. Licheng, Automatic parameters selection for SVM based on GA, in *Intelligent Control and Automation, 2004. WCICA 2004. Fifth World Congress on*, vol. 2. IEEE, pp. 1869–1872 (2004)
13. K.B. Duan, S.S. Keerthi, Which is the best multiclass SVM method? An empirical study, in *International Workshop on Multiple Classifier Systems*. Springer, pp. 278–285 (2005)
14. C. Fernando, D. Banarse, C. Blundell, Y. Zwols, D. Ha, A.A. Rusu, A. Pritzel, D. Wierstra, Pathnet: Evolution channels gradient descent in super neural networks. arXiv preprint arXiv:1701.08734 (2017)
15. M. Feurer, A. Klein, K. Eggensperger, J. Springenberg, M. Blum, F. Hutter, Efficient and robust automated machine learning, in *Advances in Neural Information Processing Systems*, pp. 2962–2970 (2015)
16. D. Floreano, P. Dürr, C. Mattiussi, Neuroevolution: from architectures to learning. Evol. Intell. **1**(1), 47–62 (2008)
17. F. Gomez, J. Schmidhuber, R. Miikkulainen, Accelerated neural evolution through cooperatively coevolved synapses. J. Mach. Learn. Res. **9**(May), 937–965 (2008)
18. I. Goodfellow, Y. Bengio, A. Courville, Y. Bengio, *Deep Learning*, vol. 1 (MIT Press, Cambridge, 2016)
19. I.J. Goodfellow, Y. Bulatov, J. Ibarz, S. Arnoud, V. Shet, Multi-digit number recognition from street view imagery using deep convolutional neural networks. arXiv preprint arXiv:1312.6082 (2013)
20. B. Graham, Fractional max-pooling. arXiv preprint arXiv:1412.6071 (2014)
21. I. Guyon, K. Bennett, G. Cawley, H.J. Escalante, S. Escalera, T.K. Ho, N. Macia, B. Ray, M. Saeed, A. Statnikov, et al.: Design of the 2015 ChaLearn AutoML challenge, in *Neural Networks (IJCNN), 2015 International Joint Conference on*. IEEE, pp. 1–8 (2015)
22. I. Guyon, I. Chaabane, H.J. Escalante, S. Escalera, D. Jajetic, J.R. Lloyd, N. Macià, B. Ray, L. Romaszko, M. Sebag, et al.: A brief review of the ChaLearn AutoML challenge: any-time any-dataset learning without human intervention, in *Workshop on Automatic Machine Learning*, pp. 21–30 (2016)
23. N. Hansen, A. Ostermeier, Completely derandomized self-adaptation in evolution strategies. Evol. Comput. **9**(2), 159–195 (2001)
24. S.A. Harp, T. Samad, A. Guha, Designing application-specific neural networks using the genetic algorithm, in *Advances in Neural Information Processing Systems*, pp. 447–454 (1990)
25. K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778 (2016)
26. Á.B. Jiménez, J.L. Lázaro, J.R. Dorronsoro, Finding optimal model parameters by deterministic and annealed focused grid search. Neurocomputing **72**(13–15), 2824–2832 (2009)
27. J.Y. Jung, J.A. Reggia, Evolutionary design of neural network architectures using a descriptive encoding language. IEEE Trans. Evol. Comput. **10**(6), 676–688 (2006)
28. J.D. Kelly Jr., L. Davis, A hybrid genetic algorithm for classification, in *Proceedings of the 12th International Joint Conference on Artificial Intelligence, Sydney, Australia*, ed. by J. Mylopoulos, R. Reiter (Morgan Kaufmann, San Francisco, 1991), pp. 645–650. http://ijcai.org/Proceedings/91-2/Papers/006.pdf
29. D. Khritonenko, V. Stanovov, E. Semenkin, Applying an instance selection method to an evolutionary neural classifier design, in *IOP Conference Series: Materials Science and Engineering*, vol. 173. IOP Publishing, p. 012007 (2017)
30. H. Kitano, Designing neural networks using genetic algorithms with graph generation system. Complex Syst. **4**(4), 461–476 (1990)
31. B. Komer, J. Bergstra, C. Eliasmith, Hyperopt-sklearn: automatic hyperparameter configuration for scikit-learn, in *ICML Workshop on AutoML* (2014)
32. A. Krizhevsky, G. Hinton, Learning multiple layers of features from tiny images (2009)
33. Y. LeCun, L. Bottou, Y. Bengio, P. Haffner, Gradient-based learning applied to document recognition. Proc. IEEE **86**(11), 2278–2324 (1998)

34. F.H.F. Leung, H.K. Lam, S.H. Ling, P.K.S. Tam, Tuning of the structure and parameters of a neural network using an improved genetic algorithm. IEEE Trans. Neural Netw. **14**(1), 79–88 (2003)
35. I. Loshchilov, F. Hutter, CMA-ES for hyperparameter optimization of deep neural networks. arXiv preprint arXiv:1604.07269 (2016)
36. N. Lourenço, F. Assunção, F.B. Pereira, E. Costa, P. Machado, Structured grammatical evolution: a dynamic approach, in *Handbook of Grammatical Evolution*, ed. by C. Ryan, M. O'Neill, J. Collins (Springer, Berlin, 2018). https://doi.org/10.1007/978-3-319-78717-6
37. N. Lourenço, F.B. Pereira, E. Costa, Unveiling the properties of structured grammatical evolution. Genet. Program. Evol. Mach. **17**(3), 251–289 (2016)
38. R. Miikkulainen, J. Liang, E. Meyerson, A. Rawal, D. Fink, O. Francon, B. Raju, A. Navruzyan, N. Duffy, B. Hodjat, Evolving deep neural networks. arXiv preprint arXiv:1703.00548 (2017)
39. J.F. Miller, Cartesian genetic programming, in *Cartesian Genetic Programming*. Springer, pp. 17–34 (2011)
40. J. Močkus, On bayesian methods for seeking the extremum, in *Optimization Techniques IFIP Technical Conference*. Springer, pp. 400–404 (1975)
41. D.E. Moriarty, R. Miikkulainen, Forming neural networks through efficient and adaptive coevolution. Evol. Comput. **5**(4), 373–399 (1997)
42. G. Morse, K.O. Stanley, Simple evolutionary optimization can rival stochastic gradient descent in neural networks, in *Proceedings of the 2016 on Genetic and Evolutionary Computation Conference*. ACM, pp. 477–484 (2016)
43. Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, A.Y. Ng, Reading digits in natural images with unsupervised feature learning, in *NIPS Workshop on Deep Learning and Unsupervised Feature Learning*, vol. 2011, p. 5 (2011)
44. M. O'Neil, C. Ryan, Grammatical evolution, in *Grammatical Evolution*. Springer, pp. 33–47 (2003)
45. F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, E. Duchesnay, Scikit-learn: machine learning in Python. J. Mach. Learn. Res. **12**, 2825–2830 (2011)
46. A. Radi, R. Poli, Discovering efficient learning rules for feedforward neural networks using genetic programming, in *Recent Advances in Intelligent Paradigms and Applications*. Springer, pp. 133–159 (2003)
47. E. Real, S. Moore, A. Selle, S. Saxena, Y.L. Suematsu, Q. Le, A. Kurakin, Large-scale evolution of image classifiers. arXiv preprint arXiv:1703.01041 (2017)
48. M. Rocha, P. Cortez, J. Neves, Evolution of neural networks for classification and regression. Neurocomputing **70**(16), 2809–2816 (2007)
49. B. Schuller, S. Reiter, G. Rigoll, Evolutionary feature generation in speech emotion recognition, in *2006 IEEE International Conference on Multimedia and Expo*. IEEE, pp. 5–8 (2006)
50. P. Sermanet, S. Chintala, Y. LeCun, Convolutional neural networks applied to house numbers digit classification, in *2012 21st International Conference on Pattern Recognition (ICPR)*. IEEE, pp. 3288–3291 (2012)
51. P.Y. Simard, D. Steinkraus, J.C. Platt et al., Best practices for convolutional neural networks applied to visual document analysis. ICDAR **3**, 958–962 (2003)
52. K. Simonyan, A. Zisserman, Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556 (2014)
53. J. Snoek, H. Larochelle, R.P. Adams, Practical Bayesian optimization of machine learning algorithms, in *Advances in Neural Information Processing Systems*, pp. 2951–2959 (2012)
54. J. Snoek, O. Rippel, K. Swersky, R. Kiros, N. Satish, N. Sundaram, M. Patwary, M. Prabhat, R. Adams, Scalable Bayesian optimization using deep neural networks, in *International Conference on Machine Learning*, pp. 2171–2180 (2015)
55. K. Soltanian, F.A. Tab, F.A. Zar, I. Tsoulos, Artificial neural networks generation using grammatical evolution, in *2013 21st Iranian Conference on Electrical Engineering (ICEE)*. IEEE, pp. 1–5 (2013)
56. K.O. Stanley, D.B. D'Ambrosio, J. Gauci, A hypercube-based encoding for evolving large-scale neural networks. Artif. Life **15**(2), 185–212 (2009)
57. K.O. Stanley, R. Miikkulainen, Evolving neural networks through augmenting topologies. Evol. Comput. **10**(2), 99–127 (2002)
58. V. Stanovov, E. Semenkin, O. Semenkina, Instance selection approach for self-configuring hybrid fuzzy evolutionary algorithm for imbalanced datasets, in *International Conference in Swarm Intelligence*. Springer, pp. 451–459 (2015)

59. M. Suganuma, S. Shirakawa, T. Nagao, A genetic programming approach to designing convolutional neural network architectures, in *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO'17*. ACM, New York, NY, USA, pp. 497–504 (2017). https://doi.org/10.1145/3071178.3071229
60. C. Thornton, F. Hutter, H.H. Hoos, K. Leyton-Brown, Auto-weka: Combined selection and hyperparameter optimization of classification algorithms, in *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, pp. 847–855 (2013)
61. A.J. Turner, J.F. Miller, Cartesian genetic programming encoded artificial neural networks: a comparison using three benchmarks, in *Proceedings of the 15th Annual Conference on Genetic and Evolutionary Computation*. ACM, pp. 1005–1012 (2013)
62. P. Verbancsics, J. Harguess, Image classification using generative neuro evolution for deep learning in *2015 IEEE Winter Conference on Applications of Computer Vision (WACV)*. IEEE, pp. 488–493 (2015)
63. D. Whitley, T. Starkweather, C. Bogart, Genetic algorithms and neural networks: optimizing connections and connectivity. Parallel Comput. **14**(3), 347–361 (1990)
64. I.H. Witten, E. Frank, M.A. Hall, C.J. Pal, *Data Mining: Practical Machine Learning Tools And Techniques* (Morgan Kaufmann, San Francisco, 2016)
65. H. Xiao, K. Rasul, R. Vollgraf, Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms (2017)
66. B. Xue, M. Zhang, W.N. Browne, X. Yao, A survey on evolutionary computation approaches to feature selection. IEEE Trans. Evol. Comput. **20**(4), 606–626 (2016)
67. X. Yao, Evolving artificial neural networks. Proc. IEEE **87**(9), 1423–1447 (1999)
68. J. Yu, B. Bhanu, Evolutionary feature synthesis for facial expression recognition. Pattern Recognit. Lett. **27**(11), 1289–1298 (2006)